Smart Builder Enterprise Resource Planning

Backend Architecture Explanation Document

PREPARED FOR

ADVANCE DATABASE MANAGEMENT SYSTEM (ADMBS)

PREPARED BY

RANA ABDUL MOEEZ	BSCS24106
AHMED NADEEM	BSCS24129
ROHAIL ASHRAF	BSCS24090

1. System Architecture & Request Lifecycle

The Construction Management System backend is a Node.js REST API providing data services to a React frontend. It manages projects, clients, human resources, materials, equipment, subcontractors and finance across multiple user roles. The project is the central entity, managed by a Project Manager who requests resources through relevant department managers. All financial actions flow through the Finance Manager.

1.1 Technology Stack

Technology Stack		
Runtime	Node.js 18+	Non-blocking, event-driven I/O suited for concurrent REST API requests.
Framework	Express.js	Minimal HTTP framework handling routing, middleware chaining and error propagation.
Database	MySQL	Relational database for structured construction data requiring multi-table joins and transactions.
Query Layer	mysql2	Raw SQL with built-in connection pool. No ORM used.
Auth / Token	jsonwebtoken	Stateless JWT bearer tokens signed with a server-side secret key.
Password Hash	crypto (Node.js) + SHA-256	Built-in Node.js crypto module. Passwords hashed with SHA-256 before storage — no third-party bcrypt dependency.
Module System	ES Modules	import/export enforced via type: module in package.json.
Error Handling	Middleware	Centralised ErrorHandler — all thrown errors from controllers and services funnel to one place.

1.2 Layered Architecture

The backend follows a strict five-layer separation of concerns. Each layer communicates only with the layer directly adjacent to it.

Layer	Files	Responsibility	Example
Routes	*_routes.js	Register HTTP method and URL path. Attach middleware. Pass to controller.	auth_routes.js, admin_routes.js, etc.
Middleware	auth.middleware.jsr bac.middleware.js	Verify JWT, decode user payload, enforce role permissions before the controller runs.	verifyToken, allowRoles(...)
Controllers	*_controller.js	Extract req.body / req.params. Call service. Send HTTP response. Never writes SQL.	project.controller.js, etc.

Services	*_services.js	All business logic. Runs SQL queries, manages transactions, enforces domain rules.	finance.services.js, etc.
Database	db.js (pool)	mysql2 connection pool. pool.query() for simple queries; pool.getConnection() for transactions.	Shared across all services.

1.3 Request Lifecycle — Step by Step

Every API call follows the same path. The example below traces POST /api/v1/material/request submitted by a Project Manager.

- React frontend sends POST /api/v1/material/request with Authorization: Bearer <token> and body { projectId, categoryId, quantity }.
- Express matches the path to material_routes.js. The route definition chains verifyToken then the controller.
- verifyToken extracts the raw token, calls jwt.verify(). On success it sets req.user = { id, role } and calls next(). On failure it returns 401.
- allowRoles('Project Manager') checks req.user.role. If the role is not permitted it returns 403. Otherwise it calls next().
- CreateMaterialRequest controller extracts projectId, categoryId, quantity from req.body and userId from req.user.id.
- The service function validates inputs, executes the SQL INSERT into material_requests, and returns the new record.
- The controller sends res.status(201).json({ success: true, data: result }) back to the frontend.
- Any uncaught throw from steps 4–6 is forwarded to the centralised ErrorHandler middleware which formats and returns the error response.

1.4 Module Responsibility Map

Module Ownership	
auth	Login, JWT issuance, token verification, password change.
admin	Director operations: create projects, set budgets, update project status.
client	Client records and per-client lookups for projects, payments, and invoices.
project	All-project queries, budget overview, dashboard aggregates, status updates.
pm	PM-scoped project view, progress log management, unified resource request gateway (/request/:type).
hr	Labour inventory, HR request creation and approval, worker allocation and release.
material	Material stock, supplier and shipment management, inventory batch queries, allocation approval.

equipment	Equipment inventory (owned vs rented), request approval, project allocation.
finance	Expense and revenue ledger, pending payment/invoice queues, approval actions.
subcontractor	Subcontractor request creation, SM approval, contract price assignment.

2. Authentication Flow & RBAC Enforcement

Security is handled in two sequential stages. Authentication confirms identity by validating the JWT token. Authorisation confirms the identity holds a role permitted on the requested route. Both run as Express middleware before any controller code executes.

2.1 Login & Token Issuance (crypto + SHA-256)

The login endpoint is the only public route. The system uses Node.js's built-in crypto module with SHA-256 for password hashing.

1. Frontend sends POST /api/auth/login with { username, password }.
2. The auth service queries: SELECT * FROM users WHERE username = ?
3. If no user is found, a 401 error is returned immediately.
4. The submitted password is hashed using crypto.createHash('sha256').update(password).digest('hex'). The result is compared against the stored hash using a constant-time comparison to prevent timing attacks.
5. On a successful match, jwt.sign({ id, role }, SECRET_KEY, { expiresIn: '24h' }) creates a signed token. The payload contains only the user's id and role.
6. The token and a minimal user object are returned inside { success: true, data: { token, user } }. The frontend attaches this as Authorization: Bearer <token> on all subsequent requests.

```
// SHA-256 hashing with Node.js crypto (no external dependency)
import crypto from 'crypto';

// On registration, hash before storing
const hash = crypto.createHash('sha256').update(password).digest('hex');

// On login, hash the submission and compare
const submitted = crypto.createHash('sha256').update(password).digest('hex');
if (submitted !== storedHash) {
      const error =  new Error('Invalid credentials');
      Error.status = 401;
      throw error;
}
```

2.2 Token Verification Middleware

Every protected route passes through verifyToken before the controller runs.

- Reads the Authorization header, strips the Bearer prefix, extracts the raw token string.
- Calls jwt.verify(token, SECRET_KEY). This checks both the signature and the expiry in one operation.
- If verification fails for any reason, returns 401 Unauthorized. The controller is never reached.
- If verification succeeds, attaches the decoded payload as req.user = { id, role } and calls next().

2.3 Role-Based Access Control

The **allowRoles** middleware is a higher-order function: it accepts an array of permitted roles and returns an Express middleware function. This makes route definitions declarative and self-documenting.

```
// rbac.middleware.js
export const allowRoles = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ message: 'Access denied' });
  next();
};

// Per-route usage (admin_routes.js)
router.get('/projects', verifyToken, allowRoles('Director'), GetAllProjects);

// Router-level blanket enforcement (finance_routes.js)
router.use(verifyToken);
router.use(allowRoles('Finance Manager'));
```

2.4 RBAC Matrix — All Endpoints

Endpoint	Method	Role Required
/auth/login	POST	public
/auth/verify	GET	Any authenticated
/auth/change-password	PUT	Any authenticated
/admin/projects	GET	Director
/admin/projects	POST	Director
/admin/projects/:id/status	PATCH	Director, Project Manager
/admin/dashboard-overview	GET	Director
/clients, /clients/:id	GET	Any authenticated
/clients/:id/projects/payments/invoices	GET	Any authenticated
/clients	POST	Any authenticated
/equipment/all /rented /owned	GET	Any authenticated
/equipment/requests/pending	GET	Any authenticated
/equipment/approve	POST	Any authenticated
/finance/expenses	GET	Finance Manager
/finance/revenues	GET	Finance Manager
/finance/payments/pending	GET	Finance Manager
/finance/invoices/pending	GET	Finance Manager
/finance/payment/approve/:id	POST	Finance Manager
/finance/invoice/approve/:id	POST	Finance Manager

/hr/resources	GET	HR Manager
/hr/requests/pending	GET	HR Manager
/hr/request /approve /free	POST	HR Manager
/material/stock /shipments /suppliers	GET	Any authenticated
/material/requests/pending	GET	Any authenticated
/material/inventory-batches	GET	Any authenticated
/material/request	POST	Any authenticated
/material/shipment/manual	POST	Any authenticated
/material/approve-batch	POST	Any authenticated
/pm/projects /projects/:id	GET	Project Manager
/pm/project/logs/:projectId	GET	Project Manager, Director
/pm/log	POST	Project Manager
/pm/request/:type	POST	Project Manager
/subcontractor/request	POST	Any authenticated
/subcontractor/requests/pending	GET	Any authenticated
/subcontractor/list/:categoryId	GET	Any authenticated
/subcontractor/approve	POST	Any authenticated

3. Transaction Flows & Rollback Conditions

3.1 Why Transactions Matter

A database transaction is a group of SQL statements that must all succeed or all fail together. Without transactions, a partial failure for example a network drop after an **INSERT** but before an **UPDATE** leaves the database in an inconsistent state. In this system every multi-table operation is wrapped in an explicit transaction using mysql2's connection-level API.

```
const conn = await pool.getConnection();
await conn.beginTransaction();
try {
  await conn.query( /* step 1 */ );
  await conn.query( /* step 2 */ );
  await conn.commit();
} catch (err) {
  await conn.rollback();
  throw err;
} finally {
  conn.release();   // ALWAYS return the connection to the pool
}
```

3.2 Project Creation Transaction

Creating a project is a two-step atomic operation. A project without a budget or a budget without a project would break financial reporting.

- **INSERT** into projects: project_name, director_id, manager_id, client_id, start_date, status = 'pending'.
- Capture insertId from the result as the new project_id.
- **INSERT** into project_budgets: project_id (FK), labour_cost, material_cost, equipment_rent, subcontractor_cost.
- **COMMIT** — both records exist and are linked.

Rollback condition: any failure in step 3 rolls back the projects INSERT. No orphaned project record is left in the database.

3.3 Project Completion Transaction

When a project is marked as complete, workers and equipment must be released atomically. If any part of the cleanup fails, the project must not be marked complete, otherwise resources would appear occupied indefinitely.

The required state changes are:

```
When project status → 'completed'
→ All allocated workers become available again
→ All open HR_Allocation records receive an end_date = TODAY
→ All allocated equipment is released (status → 'Available')
```

```
→ All open Equipment_Allocation records receive an end_date = TODAY
```

All four updates must happen atomically. The transaction steps are:

- **UPDATE** projects SET status = 'completed', end_date = TODAY WHERE id = projectId.
- **UPDATE** Skilled_Labour SET status = 'Free' WHERE hr_id IN (SELECT hr_id FROM HR_Allocation WHERE project_id = projectId AND end_date IS NULL).
- **UPDATE** Unskilled_Labour SET status = 'free' WHERE hr_id IN (same subquery).
- **UPDATE** HR_Allocation SET end_date = TODAY WHERE project_id = projectId AND end_date IS NULL.
- **UPDATE** Equipment SET status = 'Available' WHERE id IN (SELECT equipment_id FROM Equipment_Allocation WHERE project_id = projectId AND end_date IS NULL).
- **UPDATE** Equipment_Allocation SET end_date = TODAY WHERE project_id = projectId AND end_date IS NULL.
- **COMMIT**.

Rollback condition: if any of the six UPDATE statements fail, all changes roll back. The project status remains unchanged and all resources remain as they were.

4. Raw SQL & Connection Pooling Strategy

4.1 Why Raw SQL over an ORM

The system uses mysql2 with hand-written SQL queries instead of an ORM such as Sequelize or Prisma. Construction data is relational and query-heavy. Dashboard endpoints join four to six tables with aggregation functions. Raw SQL provides complete control over the query structure. Explicit transaction management using beginTransaction(), commit() and rollback() is transparent and easy to trace. There is no migration overhead because queries work directly against the existing schema.

4.2 Connection Pooling with mysql2

Establishing a new database connection takes 5–30ms. Without pooling a busy API would exhaust MySQL connections or add significant per-request latency. The system uses mysql2's built-in pool configured in a shared db.js module imported by every service file.

Connection Pool Configuration			
connectionLimit	INT	10	Maximum simultaneous open connections to MySQL.
waitForConnections	BOOL	true	Queue new requests when all 10 connections are busy, rather than throwing immediately.
queueLimit	INT	0	No cap on the waiting queue. Should be set to a finite value in production.
pool.query()	—	—	For single non-transactional queries. Pool manages borrow and return automatically.

pool.getConnection()	—	—	For transactions. Connection must be released in the finally block via conn.release().